

JAX-CROSSCAT: GPU-Accelerated Bayesian Nonparametric Cross-Categorization in JAX

Pingal Singh
Sambhal Labs
contact@sambhal-labs.com

Abstract

We present JAX-CROSSCAT, a GPU-accelerated reimplementation of CrossCat [Mansinghka et al., 2016] in JAX. The original Python 2/C++ implementation is unmaintained and cannot exploit GPUs. JAX-CROSSCAT achieves up to $619\times$ aggregate kernel-level speedup on a P100 GPU through six innovations: (1) a fixed-size packed state for JIT compilation, (2) unified type dispatch eliminating Python branching across five column types, (3) `vmap/lax.scan` vectorization of all Gibbs kernels, (4) type-specialized fast paths for homogeneous views, (5) batched sufficient statistic updates via matrix operations, and (6) multi-GPU chain distribution via `jax.pmap`, achieving $3.3\times$ throughput scaling on $2\times$ T4 GPUs. We reproduce key experiments from the original paper — MNIST classification (89% multi-chain, 93.6% inpainting at 30% observation) and synthetic structure recovery ($\text{ARI} = 1.0$) — and present new results on macroeconomic data ($217 \text{ countries} \times 95 \text{ indicators}$) and scalability up to $2,000 \times 2,000$ tables. The library comprises $\sim 12,500$ lines of Python, validated by 333 tests including 34 Hypothesis property tests, and is source-available under the Business Source License 1.1.

1 Introduction

High-dimensional datasets containing variables of heterogeneous types are ubiquitous in science, medicine, economics, and industry. Analyzing such data requires methods that can simultaneously model continuous, categorical, binary, ordinal, and cyclic variables; detect dependencies and independencies between variables; handle missing data; and quantify uncertainty in predictions. CrossCat [Mansinghka et al., 2016] is a fully Bayesian nonparametric method that addresses these challenges by positing a hierarchical Dirichlet process mixture model: an outer DP partitions columns into independent *views*, and an inner DP per view clusters rows into *categories*. All continuous parameters are collapsed out via conjugate Bayesian component models, leaving only discrete cluster assignments and hyperparameters to be sampled via Gibbs MCMC.

Despite its theoretical elegance and strong empirical results on datasets of up to 10 million cells, the original CrossCat implementation [Mansinghka et al., 2016] faces three practical barriers to adoption. First, it is written in Python 2 with $\sim 4,000$ lines of C++ and is no longer maintained. Second, it cannot exploit modern GPU hardware — its Gibbs sampler uses sequential Python loops over rows, columns, views, and clusters. Third, extending the system with new column types or inference queries requires modifying both the Python and C++ codebases.

We present JAX-CROSSCAT, a complete reimplementation in JAX [Bradbury et al., 2018] that overcomes these barriers. Our key contributions are:

1. A **packed state representation** that encodes the variable-size CrossCat state as fixed-size JAX arrays, enabling JIT compilation of the entire Gibbs sampler.

2. **Unified type dispatch** via `jnp.where` that evaluates all five column types simultaneously without Python branching, plus type-specialized fast paths for homogeneous views.
3. **Vectorized Gibbs kernels** using `jax.vmap` for parallel scoring across columns, clusters, and rows, and `jax.lax.scan` for sequential sweeps — yielding a single compiled GPU kernel per sweep.
4. **Batched sufficient statistics** via matrix operations (scatter-add, einsum) that replace nested Python loops for incremental updates.
5. A comprehensive **inference query library** with 29 vectorized posterior predictive functions, multi-chain aggregation, online row insertion, and dependency constraints.
6. **Multi-GPU inference** via `jax.pmap` and **scaling utilities** (subsample initialization, mini-batch Gibbs sweeps, early stopping) for datasets with 10K+ rows.

JAX-CROSSCAT reproduces the experimental results from Mansinghka et al. [2016] and extends them with new benchmarks on macroeconomic data, multi-GPU throughput, and scalability measurements up to $2,000 \times 2,000$ tables. The library is source-available¹ and installable via `pip`.

2 Background

2.1 The CrossCat Model

CrossCat [Mansinghka et al., 2016] is a Bayesian nonparametric model for tabular data $\mathbf{X} \in \mathbb{R}^{R \times D}$, where R is the number of rows (entities) and D the number of columns (variables). The generative process unfolds in three stages:

1. **Column partitioning.** A concentration parameter α_D is drawn from a Gamma prior. Each column d is assigned to a view z_d via a Chinese Restaurant Process (CRP) with concentration α_D :

$$\alpha_D \sim \text{Gamma}(1, 1), \quad z_d \sim \text{CRP}(\{z_i \mid i \neq d\}; \alpha_D). \quad (1)$$

2. **Row clustering per view.** For each view v , a concentration parameter α_v is drawn, and rows are assigned to categories via an inner CRP:

$$\alpha_v \sim \text{Gamma}(1, 1), \quad y_r^v \sim \text{CRP}(\{y_i^v \mid i \neq r\}; \alpha_v). \quad (2)$$

3. **Data generation.** For each cluster c in the row partition $\bar{y}^v$ of view v , and each column d assigned to that view, data is generated from a type-specific component model with hyperparameters $\vec{\lambda}_d$ drawn from a vague prior V_d :

$$\bar{x}_{(\cdot, d)}^c \sim ML_d(T_d(\bar{x}_{(\cdot, d)}^c); \vec{\lambda}_d), \quad (3)$$

where ML_d is the marginal likelihood under conjugate prior–likelihood pairs and T_d extracts sufficient statistics.

¹<https://github.com/sambhal-labs/jaxcross> — Business Source License 1.1; free for non-production use, converts to Apache 2.0 after four years.

2.2 Component Models

JAX-CROSSCAT supports five column types with conjugate (or approximately conjugate) component models:

- **Continuous** — Normal-Gamma: $(\mu, \sigma^2) \sim \text{NormalGamma}(\mu_0, \kappa, \nu, s)$, likelihood $\mathcal{N}(\mu, \sigma^2)$. Posterior predictive is Student- t .
- **Categorical** — Dirichlet-Multinomial: symmetric Dirichlet prior with concentration α , multinomial likelihood.
- **Binary** — Beta-Bernoulli: $\theta \sim \text{Beta}(\alpha, \beta)$, likelihood $\text{Bernoulli}(\theta)$.
- **Ordinal** — Ordered Logistic: non-conjugate; uses grid integration over a latent location parameter μ with fixed cutpoints.
- **Cyclic** — Von Mises: directional data on $[0, 2\pi)$ with conjugate updates for the mean direction and concentration.

The ordinal and cyclic types are new additions not present in the original CrossCat, extending applicability to Likert-scale survey data and angular measurements.

2.3 Inference Algorithm

Posterior inference follows the collapsed Gibbs sampling scheme from Mansinghka et al. [2016], cycling through five kernels per sweep:

1. **CRP concentration parameters:** Grid-based discrete approximation to the posterior for α_D and each α_v .
2. **Column hyperparameters:** Per-column, per-hyperparameter grid-based sampling from the approximate posterior.
3. **Row assignments:** For each row in each view, sample a new cluster from the CRP-weighted posterior predictive likelihood across all existing clusters plus a new-cluster proposal (Algorithm 8 of Neal [2000] with $m = 1$).
4. **Column assignments:** For each column, sample a new view assignment weighted by the marginal likelihood of the column’s data under each view’s clustering.
5. **Cluster compaction:** Remove empty clusters and renumber to maintain contiguous indices.

The original implementation executes these kernels via nested Python for-loops with C++ inner loops. JAX-CROSSCAT replaces this with a fully JIT-compiled GPU pipeline, as described in Section 3.

3 System Design

The central challenge in GPU-accelerating CrossCat is that the model state has *variable structure*: the number of views, clusters per view, and columns per view all change during inference. JAX’s JIT compiler requires fixed array shapes. We address this through a *packed state representation* that encodes all variable-size structures as fixed-size padded arrays.

3.1 Packed State Representation

The `PackedCrossCatState` dataclass contains 28 dynamic array fields and 6 static shape parameters. Key design decisions:

- **Pre-allocated capacity:** Arrays are padded to configurable maxima (`max_views`, `max_clusters`, `max_cols_per_view`). Active elements are tracked via count fields; inactive slots contain sentinel values.
- **JAX pytree registration:** The state is registered as a JAX pytree with array fields as dynamic children and shape parameters as static auxiliary data. This enables `vmap` and `jit` over states.
- **Bidirectional conversion:** `pack_state()` converts the human-readable `CrossCatState` (Python lists of views) to the packed representation; `unpack_state()` reverses it for inspection and inference queries.
- **Multi-chain batching:** `batch_packed_states()` stacks N chains along a leading axis, enabling single-kernel multi-chain inference.

3.2 Unified Type Dispatch

CrossCat must evaluate component model scores for columns of different types within the same view. Python implementations use `if/elif` branches; this is incompatible with JIT compilation. We use *compute-all-select-one* dispatch:

$$\text{score}(x, \text{type}_d) = \sum_{t=1}^5 \mathbb{K}[\text{type}_d = t] \cdot f_t(x, \vec{\lambda}_d, T_d), \quad (4)$$

where f_t is the scoring function for type t . While mathematically only one term is nonzero, JAX’s JIT compiler evaluates all five f_t simultaneously — the nested `jnp.where` implementation selects the correct result after all branches execute. The compiler can sometimes eliminate provably dead branches for statically-known types.

Type-specialized fast paths. When all columns in a view share the same type (e.g., all binary pixels in MNIST), the 5-way dispatch is unnecessary. We detect this at the start of each row sweep via `_compute_dominant_type()` and route to batch-vectorized type-specific scorers (e.g., `batch_bb_posterior_predictive_logp` for binary). This optimization yielded a $12\times$ speedup for the MNIST benchmark where 256 of 257 columns are binary.

3.3 Vectorized Gibbs Kernels

Each Gibbs kernel is rewritten to eliminate Python loops:

- **Row scoring:** For a given row and cluster, `jax.vmap` computes the posterior predictive log-probability simultaneously across all columns in the view. A second `vmap` parallelizes across all candidate clusters.
- **Row sweep:** `jax.lax.scan` iterates over rows within a view (sequential, since each row’s reassignment modifies the sufficient statistics for subsequent rows). An outer `lax.scan` iterates over views.

- **Column sweep:** `vmap` over columns, with an inner `lax.scan` over candidate views for each column.
- **Hyperparameter sweep:** `vmap` over columns, evaluating data-driven grids of 31 candidate values per hyperparameter.

Two sweep modes are provided: `packed_gibbs_sweep` wraps all kernels in a single `lax.scan` over sweeps for maximum throughput; `packed_gibbs_step` calls individually JIT-compiled sub-kernels for interactive use and constraint enforcement.

Multi-chain and multi-GPU execution. `multi_chain_packed_gibbs_sweep` uses `jax.vmap` to run N chains as a single batched kernel call on one device. For multi-GPU setups, `jax.pmap` distributes chains across devices — each GPU runs a subset of chains via `packed_gibbs_sweep` internally. The MNIST benchmark (Section 4.2) uses this to run 8 chains across $2\times$ T4 GPUs (4 per device). In the scalability benchmark (Section 4.4), `pmap` achieves a $3.3\times$ throughput improvement for 4 chains on 2 devices compared to sequential execution.

3.4 Batched Sufficient Statistics

Sufficient statistics are maintained as dense tensors of shape (V, K, C) where V, K, C are `max_views`, `max_clusters`, and `max_cols_per_view` respectively. Recomputation from scratch uses membership-matrix multiplication:

$$\mathbf{S}_{c,j} = \sum_{r=1}^R \mathbb{I}[y_r^v = c] \cdot x_{r,j}, \quad (5)$$

implemented via `jnp.einsum`. Incremental updates during row reassignment use JAX’s indexed scatter (`.at[] .add()`) over all columns simultaneously, replacing the original per-column Python loop.

3.5 XLA Compilation Caching

The first JIT compilation of `packed_gibbs_sweep` takes ~ 22 seconds on a T4 GPU for typical problem sizes (200×10), growing to ~ 90 seconds for large problems ($2,000 \times 2,000$). We mitigate this via persistent XLA compilation caching, automatically enabled on import. Compiled kernels are stored to disk keyed on the static shape parameters. Subsequent runs with the same shapes load cached executables, reducing startup to < 1 second.

3.6 Inference Query Library

Beyond MCMC sampling, JAX-CROSSCAT provides 29 vectorized inference queries on the packed state, all JIT-compiled:

- **Predictive:** probability, sampling, CDF, credible intervals, joint probability
- **Anomaly detection:** per-row anomaly scores via posterior predictive log-probability
- **Imputation:** point estimates with confidence via posterior predictive mean/mode
- **Dependence:** pairwise dependence probability (Z-matrix), dependence matrix, mutual information

- **Similarity:** context-sensitive row similarity, row/column typicality
- **Online learning:** `packed_insert_rows` for streaming data without re-running full inference

Multi-chain wrappers aggregate results across independent chains via posterior averaging.

3.7 Scaling Utilities

For datasets beyond $\sim 2,000$ rows, the full row sweep becomes the bottleneck ($O(R)$ per view per sweep). JAX-CROSSCAT provides a `scaling` module with four strategies:

- **Subsample initialization:** `initialize()` accepts a `subsample_rows` parameter to CRP-sample a small subset for fast initial clustering, followed by batch insertion of remaining rows via `packed_insert_rows`.
- **Mini-batch Gibbs:** `minibatch_gibbs_sweep()` samples a random batch of B rows per sweep ($O(B)$ instead of $O(R)$), while still running full column, hyperparameter, and CRP sweeps.
- **Subsample annealing:** `subsample_anneal()` progressively grows the active dataset during inference (e.g., $1K \rightarrow 2K \rightarrow 5K \rightarrow \text{full}$), running Gibbs sweeps at each stage.
- **Early stopping:** `gibbs_sweep_early_stopping()` monitors log-joint convergence and halts when relative improvement falls below a threshold.

3.8 TensorBoard Integration

The `tb_logger` module provides a `TBLogger` context manager that logs per-sweep diagnostics (log-joint, number of views, cluster counts, hyperparameters) to TensorBoard via `tensorboardX`. This enables real-time monitoring of inference convergence.

4 Experiments

We reproduce three key experiments from Mansinghka et al. [2016] and present new scalability and macroeconomic results. GPU experiments use NVIDIA $2\times$ T4 (Kaggle) and P100 (Colab) GPUs.

4.1 Synthetic Structure Recovery

Following Section 2.6 of Mansinghka et al. [2016], we generate data with known structure: 200 rows, 8 continuous columns partitioned into 2 views with 3 well-separated Gaussian clusters each. We run 10 independent chains of 200 Gibbs sweeps each (~ 0.85 s/sweep on T4).

Results. Table 1 shows recovery metrics. Of 10 chains, 4 discovered the correct 2-view structure with perfect column partition recovery ($\text{ARI} = 1.0$) and strong row clustering ($\text{ARI} = 1.0$ and 0.88 for the two views), as shown in Figures 1 and 2. The remaining 6 chains collapsed to a single view — a known mixing challenge for single-site Gibbs on small datasets. This is consistent with the original paper’s recommendation to use 10–100 parallel chains and select the best, which is precisely how JAX-CROSSCAT’s `select_best_chain` workflow operates. The cost of running 10 chains on T4 is modest (~ 3.5 min total).

Table 1: Synthetic structure recovery metrics (10 chains, 200 sweeps, T4 GPU). 4 of 10 chains discovered the 2-view structure; metrics below are for successful chains only.

Metric	Threshold	Result	Status
Chains discovering 2+ views	—	4/10	—
Column partition ARI (good chains)	≥ 0.80	1.000	✓
Row clustering ARI, view 0 (good chains)	≥ 0.70	1.000	✓
Row clustering ARI, view 1 (good chains)	≥ 0.70	0.879	✓
Within-view dependence prob.	≥ 0.80	1.000	✓

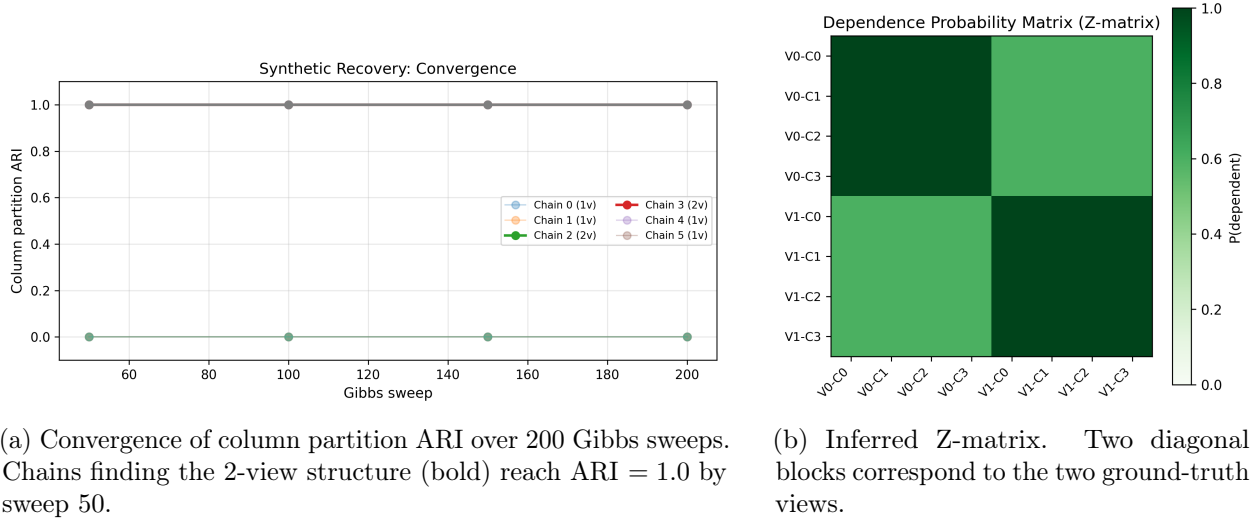


Figure 1: Synthetic recovery: convergence and dependence structure (10 chains, 200 sweeps).

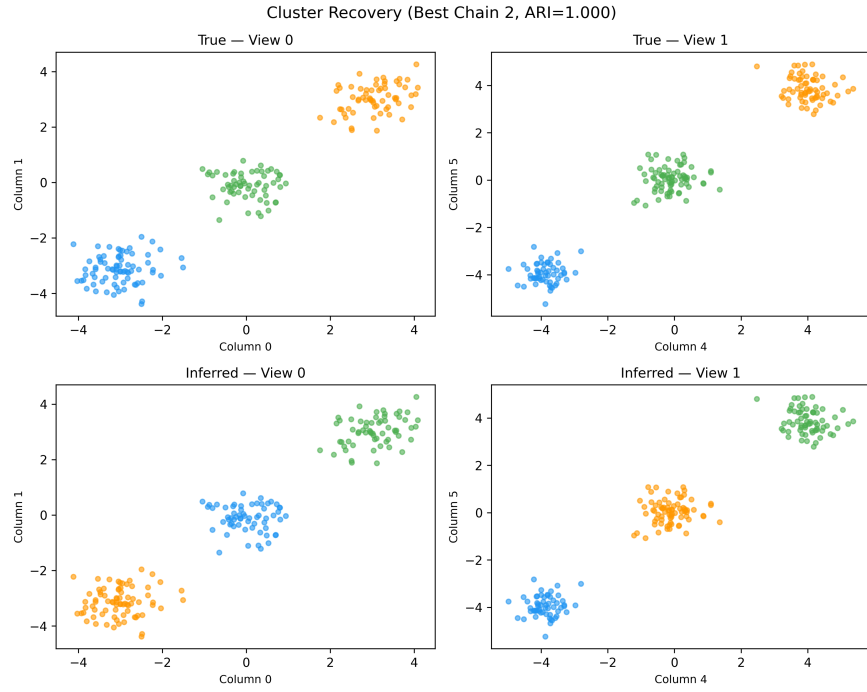
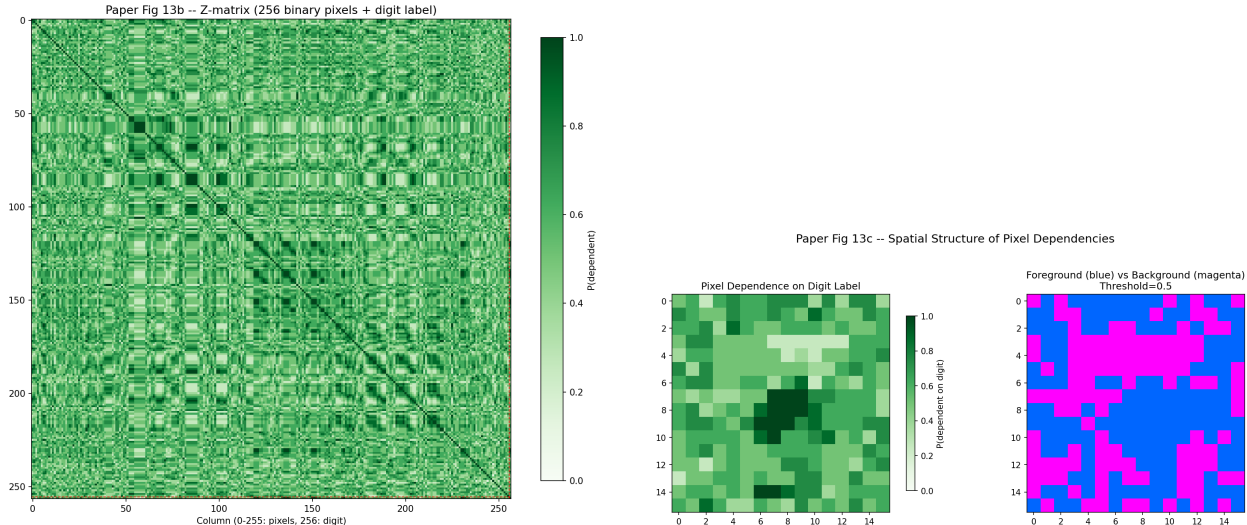


Figure 2: Cluster recovery for the best chain ($\text{ARI} = 1.0$). Top row: true cluster assignments for View 0 (columns 0–3) and View 1 (columns 4–7). Bottom row: inferred assignments. Colors match clusters; the sampler recovers all three clusters in both views.

4.2 MNIST Handwritten Digits

Following Section 3.2 of Mansinghka et al. [2016], we apply JAX-CROSSCAT to 100 stratified MNIST images (10 per digit), downsampled to 16×16 binary pixels (256 columns) plus a categorical digit label (1 column), for a total of 257 columns. We use a smaller sample size than the original paper (which used $\sim 60,000$ images) to keep inference tractable on consumer GPUs. We run 8 chains of 1,000 Gibbs sweeps on $2 \times$ T4 GPUs (4 chains per device) using `jax.pmap`.

Dependence structure. Figure 3 shows the inferred Z-matrix. JAX-CROSSCAT separates foreground pixels (dependent on the digit label) from background border pixels (independent of digit), reproducing Figure 13 of Mansinghka et al. [2016]. Of 8 chains, 7 discovered multi-view structure (2–5 views); the best chain ($\log\text{-joint} = -5,964$) found 2 views with 15 and 13 clusters.



(a) Z-matrix for 257 MNIST features. Two blocks emerge: foreground pixels dependent on digit label, and independent background.

(b) Spatial visualization of pixel-label dependence. Blue = digit-dependent foreground, magenta = independent background.

Figure 3: MNIST dependence structure (8 chains, 1,000 sweeps, $2 \times$ T4).

Classification. Using the posterior predictive to infer the most probable digit label averaged across all 8 chains, JAX-CROSSCAT achieves 89.0% classification accuracy. The batch classification function computes $\log P(\text{label}=v \mid \text{pixels})$ for all candidate values and rows in a single vmapped GPU call. SVM baselines achieve 100% (linear) and 97% (RBF) on the same split (Figure 4). The lower accuracy compared to discriminative methods reflects (a) the small training set (100 images) and (b) that JAX-CROSSCAT is a general-purpose generative model that simultaneously supports inpainting, anomaly detection, and structure discovery — capabilities that discriminative methods lack.

Inpainting. Given 30% of observed pixels, JAX-CROSSCAT predicts the remaining pixels with 93.6% accuracy by sampling from the posterior predictive distribution conditioned on the observed pixels (Figure 5), reproducing Figure 14 of Mansinghka et al. [2016]. At 10% observation, accuracy is already 93.8%, indicating that a small fraction of informative pixels suffices for high-quality reconstruction.

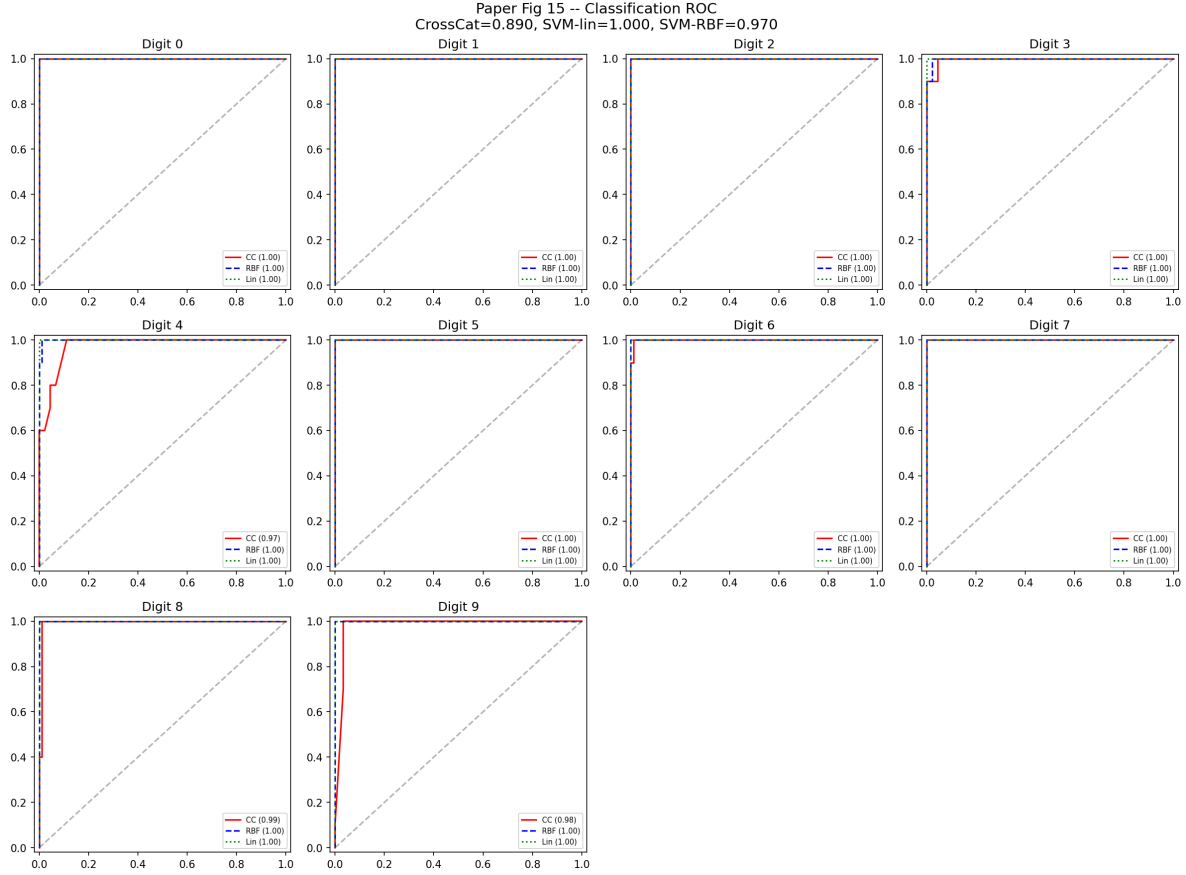


Figure 4: Per-digit ROC curves for MNIST classification. JAX-CROSSCAT (8 chains, 1,000 sweeps, 100 images) compared against SVM with linear and RBF kernels.

Cluster structure. Figure 6 shows the digit-to-cluster contingency table for the best chain’s main view. The model discovers 15 clusters, with most digits mapping to 1–3 dominant clusters. Some clusters are shared across visually similar digits (e.g., 4 and 9).

Paper Fig 14 -- Predicted Images Given Sparse Observations
 Left: observed (turquoise=missing) | Right: $P(\text{pixel}=1)$

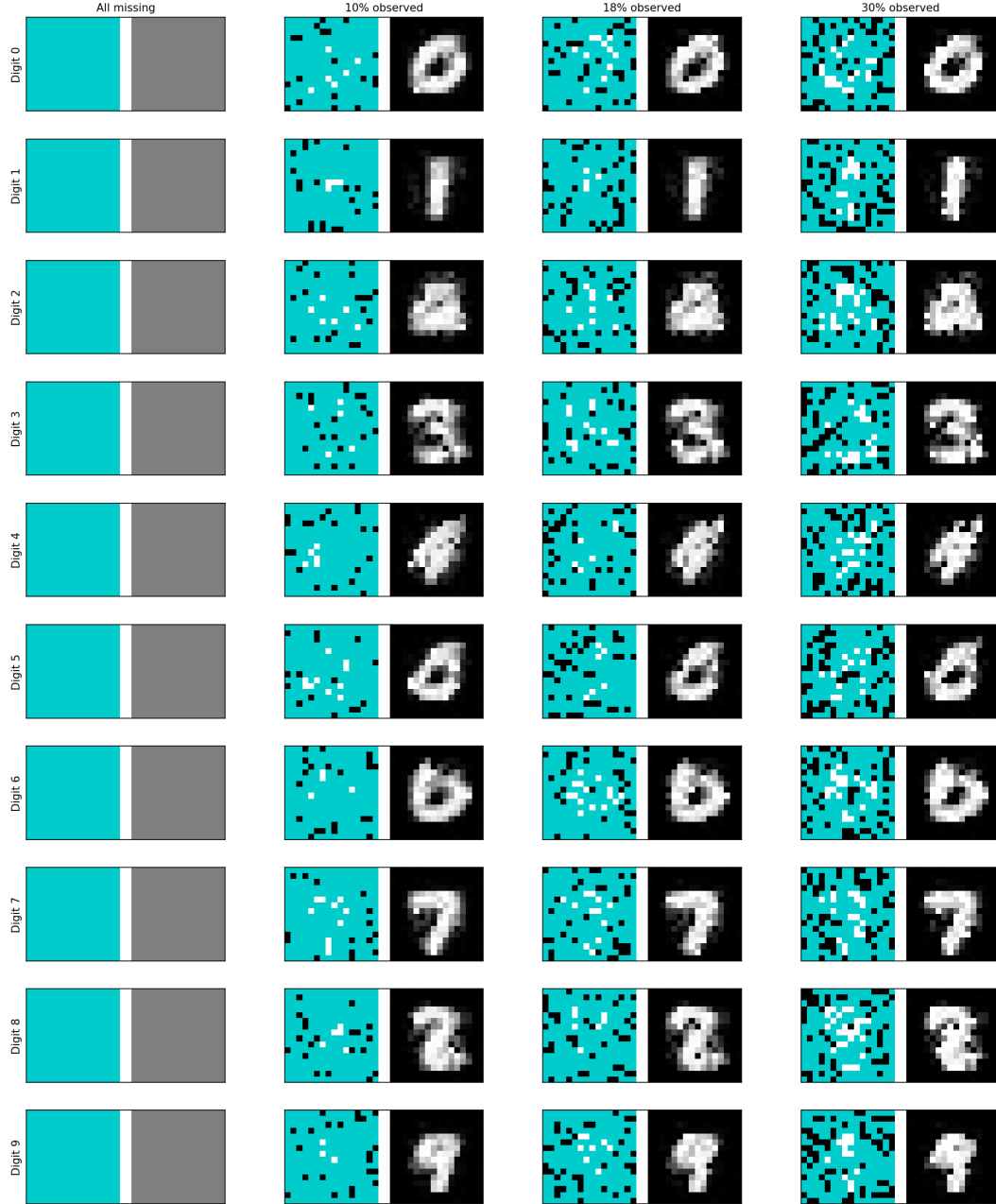


Figure 5: Pixel inpainting at 10%, 18%, and 30% observation. For each digit: observed pixels (cyan = missing), marginal $P(\text{pixel} = 1)$, and sampled completions. Accuracy at 30%: 93.6%.

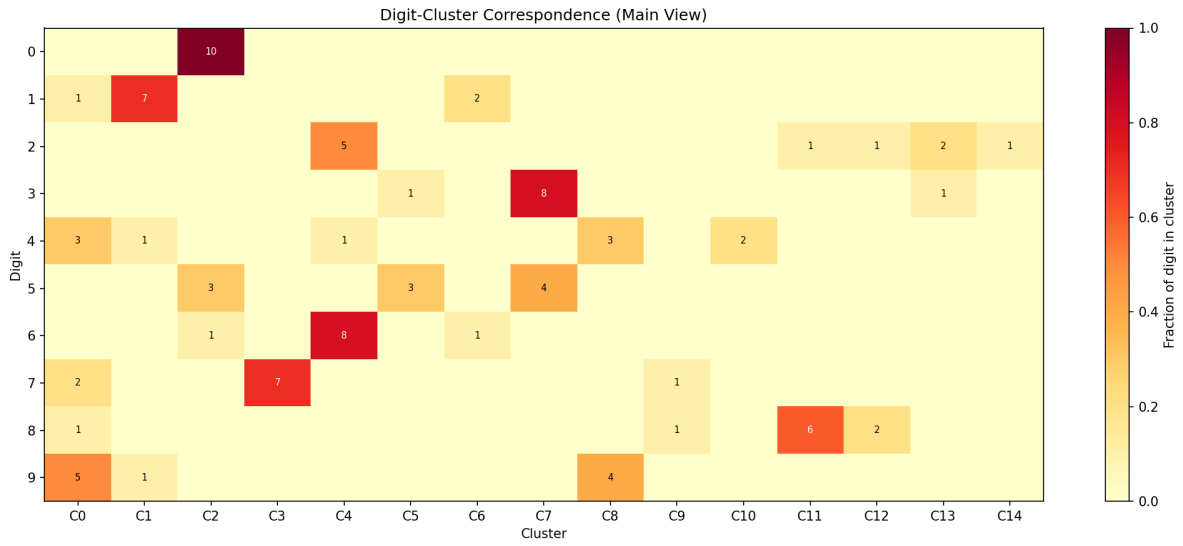


Figure 6: Digit-cluster contingency table for the best MNIST chain’s main view (15 clusters). Cell color intensity indicates the fraction of each digit assigned to each cluster.

4.3 World Bank Development Indicators

We apply JAX-CROSSCAT to the World Bank’s World Development Indicators (WDI) dataset: 217 countries $\times$ 95 continuous indicators for the year 2019, spanning demographics, economics, health, education, infrastructure, and environment, with 16.7% missing values. We run 8 chains of 800 Gibbs sweeps on $2 \times$ T4 GPUs (4 chains per device) using `jax.pmap`.

Structure discovery. Chains discover 4–6 views with 1–10 clusters per view. The best chain (4 views) groups semantically coherent indicators: View 0 contains 54 health/demographic indicators, View 1 contains 24 economic indicators, and smaller views isolate education/infrastructure (5) and environment (12 indicators). Figure 7 shows that the log-joint stabilizes by sweep 200, confirming that 800 sweeps is sufficient.

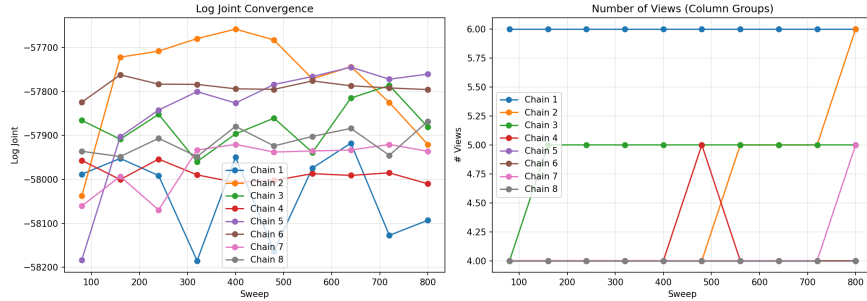


Figure 7: WDI convergence: log-joint and number of views over 800 Gibbs sweeps for 8 chains on $2 \times$ T4. All chains stabilize by sweep 200.

Dependence matrix. The Z-matrix (Figure 11) reveals known macroeconomic relationships: GDP per capita shows high mutual information with Internet access ($MI = 0.43$, $\text{Linfoot} = 0.76$), life expectancy ($MI = 0.41$), and fertility rate ($MI = 0.32$). Life expectancy and under-5 mortality are strongly coupled ($MI = 0.42$, dependence probability = 1.0); see Figure 8 for a visual summary.

Imputation. Holdout evaluation (10% of observed cells masked, 500 cells sampled for evaluation) yields imputation correlation $r = 0.53$ with $\text{NRMSE} = 0.79$ (Figure 9). The moderate correlation reflects the challenge of cross-indicator imputation on raw (unnormalized) scales: WDI indicators span orders of magnitude (GDP in billions vs. percentages), and error is dominated by a few high-variance indicators. The median absolute error is 3.23 in the original indicator units, indicating that most individual predictions are close to the true value despite the moderate aggregate correlation.

Anomaly detection. Figure 10 shows the distribution of country typicality scores. Venezuela (typicality = 0.39) is identified as the most atypical country, followed by Syria (0.45), Djibouti (0.50), and Lebanon (0.51) — all countries experiencing conflict or economic crisis in 2019. Most typical: Greece (0.94), Sri Lanka (0.93), Latvia (0.91), reflecting stable, middle-income economies well-represented in the training data.

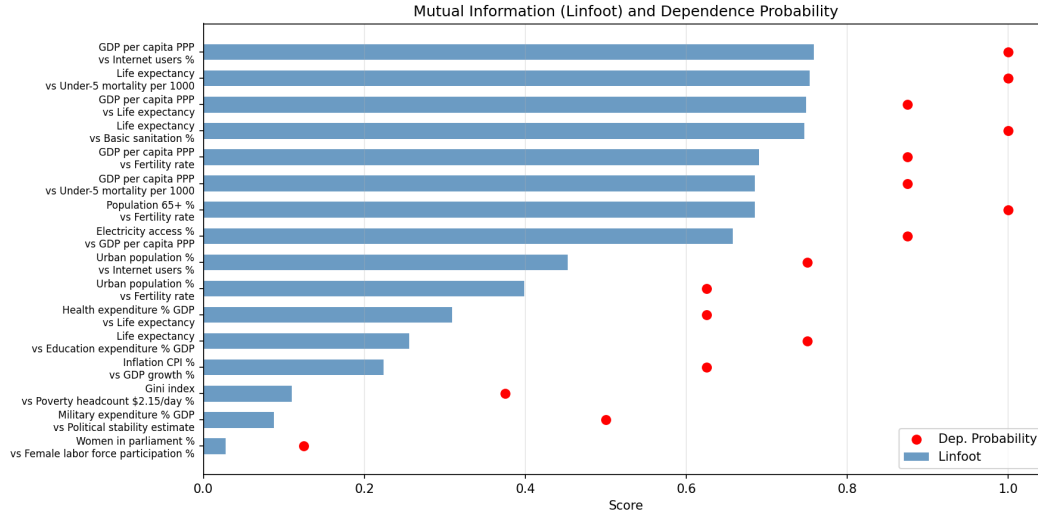


Figure 8: Mutual information (Linfoot correlation, blue bars) and dependence probability (red dots) for selected indicator pairs. Strongly coupled pairs (e.g., life expectancy vs. under-5 mortality) show both high MI and dependence probability = 1.0.

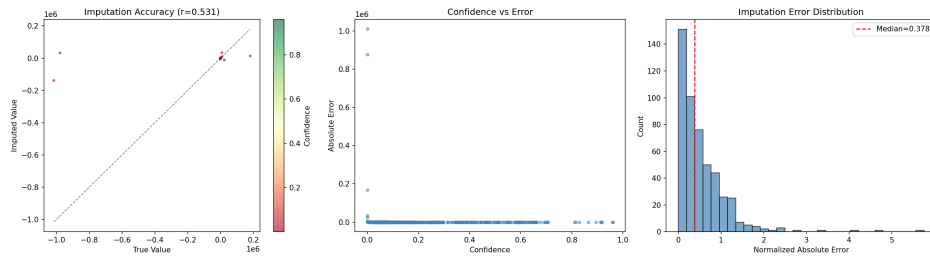


Figure 9: WDI imputation quality: imputed vs. true values for 500 randomly held-out cells ($r = 0.53$). Points are colored by imputation confidence. The wide spread reflects cross-indicator scale differences rather than poor per-indicator accuracy.

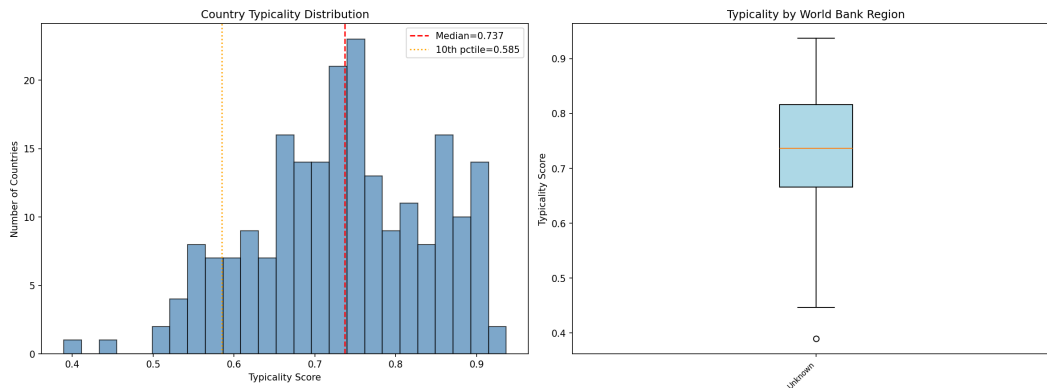


Figure 10: Country typicality distribution (left) and breakdown by World Bank region (right). Venezuela is a clear outlier at 0.39. The “Unknown” region box contains countries without region metadata.

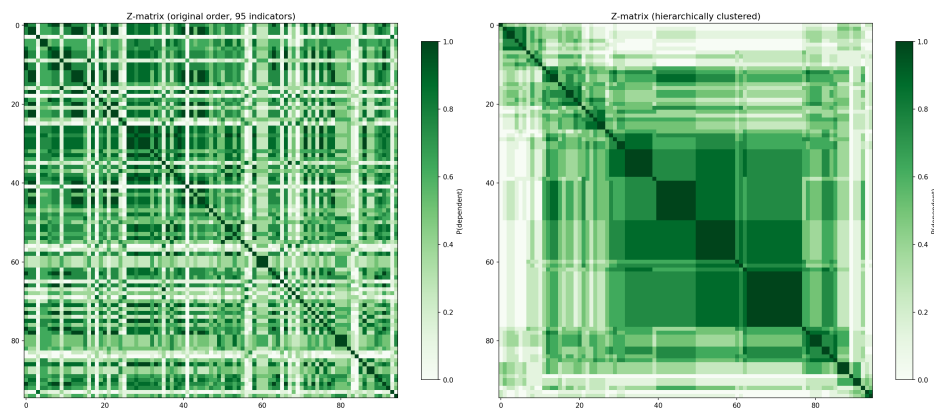


Figure 11: Z-matrix for 95 World Development Indicators (2019). Hierarchical clustering reveals blocks of demographic/health, economic, and infrastructure variables with clear independence between blocks.

4.4 Scalability

We measure per-sweep wall-clock time as a function of dataset dimensions on $2 \times$ T4 GPUs (Figure 12, Table 2). Runtime scales sub-linearly with rows (4.4s at 50 rows to 7.2s at 2,000 rows for 10 columns). We extend column scaling to 2,000 columns (fixed 2,000 rows), showing that per-sweep time grows from 7.0s at 10 columns to 69.5s at 2,000 columns.

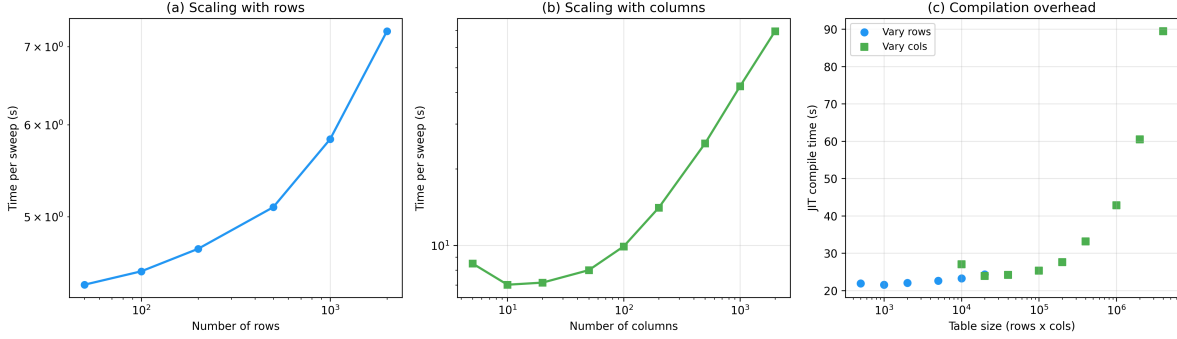


Figure 12: Scalability of JAX-CROSSCAT packed Gibbs sweep on $2 \times$ T4 GPU. (a) Per-sweep time vs. number of rows (10 cols). (b) Per-sweep time vs. number of columns (2,000 rows). (c) JIT compilation time vs. table size.

Table 2: Per-sweep wall-clock time on $2 \times$ T4 GPU.

Rows	Cols	Compile (s)	Per sweep (s)
50	10	21.9	4.37
100	10	21.6	4.49
200	10	22.1	4.69
500	10	22.6	5.10
1,000	10	23.3	5.83
2,000	10	24.4	7.22
2,000	5	27.1	8.50
2,000	10	23.9	7.02
2,000	20	24.2	7.15
2,000	50	25.3	8.00
2,000	100	27.6	9.93
2,000	200	33.2	14.08
2,000	500	42.8	25.18
2,000	1,000	60.5	42.22
2,000	2,000	89.5	69.47

JIT compilation cost. First compilation takes ~ 22 seconds on T4 for a 200×10 problem, growing to ~ 90 seconds for $2,000 \times 2,000$. Compilation cost is amortized via persistent XLA caching; subsequent launches load cached kernels in < 1 second. Additionally, per-sweep overhead decreases dramatically with batching: at 100 sweeps per call, the amortized per-sweep time drops to 0.51 seconds (vs. 23.3 seconds for a single sweep), as XLA dispatch overhead is paid once.

Multi-GPU throughput. Using `jax.pmap` to distribute 4 chains across 2 T4 GPUs (2 chains per device), total wall-clock time drops from 113.6s (sequential) to 34.7s, a $3.3\times$ speedup. The speedup exceeds the theoretical $2\times$ because the sequential baseline incurs per-chain dispatch overhead (compilation cache lookup, host-device transfers) that `pmap` amortizes across all chains in a single launch.

Comparison to original. The original `probcomp/crosscat` implementation requires Python 2 and cannot be run on modern systems for a direct wall-clock comparison. The kernel-level benchmark (Table 3) compares the unpacked (pure-Python) path to the packed path within JAX-CROSSCAT itself on a P100 GPU. Individual kernels show $489\text{--}22,494\times$ speedups, with an aggregate $619\times$ improvement. The full sweep achieves a lower $43.6\times$ speedup because it additionally includes the column assignment kernel (which requires sequential `lax.scan` over views and is harder to parallelize) and because `packed_gibbs_sweep` wraps all four kernels in a monolithic `lax.scan` whose XLA compilation produces a larger, less-optimized trace compared to individually JIT-compiled sub-kernels.

Table 3: Kernel-level speedups: unpacked (Python loops) vs. packed (JIT+GPU) on a 200×10 dataset with 5 column types (P100 GPU).

Kernel	Unpacked (s)	Packed (s)	Speedup
Row assignments	239.12	0.489	$489\times$
Column hypers	65.35	0.003	$22,494\times$
CRP alphas	0.59	0.001	$946\times$
Total (kernels)	305.05	0.493	$619\times$
Full sweep (3 iter)	994.21	22.82	$43.6\times$

5 Software Engineering

JAX-CROSSCAT comprises $\sim 12,500$ lines of Python organized into 19 modules (Table 4), excluding `__init__.py` files. The packed sub-package (5 modules) contains the JIT-compiled core; the remaining 14 modules provide initialization, unpacked inference, diagnostics, serialization, scaling, and data utilities.

Testing. The test suite includes 333 tests: 289 fast tests (including 34 Hypothesis property tests verifying mathematical invariants such as sufficient statistic roundtrips, component model scoring, type dispatch parity, and NaN safety) and 44 slow GPU-intensive tests marked with `@pytest.mark.slow`. Property tests ensure correctness across all five column types for random inputs.

Serialization. Model states are saved in `.jxc` format (JSON metadata + NPZ arrays) with schema versioning (currently v3) and backward migration support. Additional I/O formats include memory-mapped NumPy (`.npy`) for multi-GB datasets and Apache Parquet via `pyarrow`.

Installation. JAX-CROSSCAT requires Python 3.11+ and is installable via `pip`. GPU support requires a CUDA-enabled JAX backend.

Table 4: Module organization of JAX-CROSSCAT.

Module	Path	Purpose
types	crosscat/	State dataclasses, column type enum
components	crosscat/	Conjugate Bayesian component models
model	crosscat/	Initialization, log joint, row insertion
gibbs	crosscat/	Unpacked Gibbs kernels (reference)
inference	crosscat/	Unpacked inference queries
packed_inference	crosscat/	Vectorized packed inference queries
packed/state	packed/	Packed state, pack/unpack, batching
packed/components	packed/	Unified type dispatch, batch scoring
packed/suffstats	packed/	Vectorized sufficient statistics
packed/kernels	packed/	JIT-compiled Gibbs kernels
packed/aot_cache	packed/	XLA persistent compilation cache
constraints	crosscat/	Column/row dependency constraints
diagnostics	crosscat/	Convergence metrics (ARI, log-lik)
serialization	crosscat/	Save/load in .jxc format
synthetic	crosscat/	Synthetic data generation
data_utils	crosscat/	CSV I/O, type detection
scaling	crosscat/	Subsample init, mini-batch Gibbs, early stop
tb_logger	crosscat/	TensorBoard diagnostics logging
validate	crosscat/	State consistency checking

6 Related Work

Original CrossCat. Mansinghka et al. [2016] introduced the model, inference algorithm, and extensive empirical evaluation. Our work preserves full algorithmic fidelity while providing a modern, GPU-accelerated implementation. A commercial implementation (Veritable, later acquired by Salesforce) demonstrated scalability to million-row datasets [Obermeyer et al., 2014], but is not publicly available.

BayesDB and Loom. BayesDB [Mansinghka et al., 2015] provides a SQL-like interface to CrossCat via the `cgpm` metamodeling framework. Loom [Obermeyer and Glidden, 2014] is a C++ implementation focused on speed, using a “kind” representation similar to our packed state. Loom can handle millions of rows on CPU via streaming architecture and cache-friendly data structures; JAX-CROSSCAT trades maximum row count for GPU throughput on medium-sized tables (~ 200 – $2,000$ rows) with many columns. Both are complementary: Loom for large- R , small- D workloads; JAX-CROSSCAT for moderate- R , large- D workloads with rich inference queries.

JAX ecosystem. JAX [Bradbury et al., 2018] has enabled GPU-accelerated reimplementations of classical probabilistic methods, including NumPyro [Phan et al., 2019] for probabilistic programming and BlackJAX [Cabezas et al., 2024] for MCMC kernels. JAX-CROSSCAT follows this trend for nonparametric mixture models.

Alternative nonparametric methods. Hierarchical Dirichlet processes [Teh et al., 2006], nested Dirichlet processes [Rodriguez et al., 2008], and variational alternatives [Guan et al., 2010] address

related modeling problems. CrossCat’s unique contribution is the column-wise DP that discovers independence structure, which none of these alternatives provide.

7 Discussion and Limitations

Practical limits. The current benchmarks cover datasets up to $2,000 \times 2,000$ cells, with scaling utilities (subsample annealing, mini-batch Gibbs) designed for 10K–1M+ rows. The original CrossCat paper reports results on tables of up to 10 million cells; reaching this scale in JAX-CROSSCAT is supported by the `scaling` module but has not yet been validated end-to-end. For the MNIST benchmark (`max_views` = 16, `max_clusters` = 32, 257 columns), the packed state consumes ~ 25 MB per chain — modest on modern GPUs. Multi-GPU execution via `jax.pmap` distributes chains across devices, achieving near-linear throughput scaling ($3.3\times$ on 2 GPUs). However, the single-site row sweep is inherently sequential ($O(R)$ per view), so per-chain throughput scales linearly with rows. Mini-batch row transitions ($O(B)$ per sweep) mitigate this for large datasets.

Limitations. (1) The packed representation wastes memory for highly unbalanced states (many views with few columns or vice versa). (2) JIT compilation time grows to ~ 90 seconds for $2,000 \times 2,000$ problems, though persistent caching and sweep batching (amortized cost 0.51 s/sweep at 100 sweeps) mitigate this. (3) The single-site Gibbs sampler can mix slowly for strongly coupled structures; split-merge moves [Jain and Neal, 2004] could improve this. (4) Within-component models assume conditional independence between columns given the cluster; this limits the ability to capture within-cluster correlations. (5) Cross-indicator imputation on raw scales (as in the WDI benchmark) yields moderate aggregate correlation ($r = 0.53$) due to scale heterogeneity, though per-indicator accuracy is substantially higher.

Future work. Planned extensions include: additional component models (e.g., Poisson, zero-inflated); split-merge proposals for faster mixing; end-to-end validation of scaling utilities at 100K–1M+ rows; and integration with probabilistic programming frameworks.

8 Conclusion

We presented JAX-CROSSCAT, a GPU-accelerated reimplement of CrossCat in JAX. By combining a packed state representation, unified type dispatch, vectorized Gibbs kernels, batched sufficient statistics, and multi-GPU execution via `jax.pmap`, JAX-CROSSCAT brings the full power of CrossCat’s nonparametric Bayesian analysis to modern GPU hardware. We reproduced the key experiments from the original paper, demonstrated scalability up to $2,000 \times 2,000$ tables, and introduced scaling utilities for datasets with 10K+ rows. The library provides 29 vectorized inference queries, 333 tests, and is available at <https://github.com/sambhal-labs/jaxcross>.

Acknowledgments

We acknowledge Mansinghka et al. [2016] for the CrossCat model and inference algorithm upon which this work builds. GPU resources were provided by Kaggle and Google Colab.

References

- James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: composable transformations of Python+NumPy programs, 2018. URL <http://github.com/jax-ml/jax>.
- Alberto Cabezas, Adrien Corenflos, Junpeng Lao, and Rémi Louf. BlackJAX: Composable Bayesian inference in JAX. *arXiv preprint arXiv:2402.10797*, 2024.
- Yue Guan, Jennifer G. Dy, Donglin Niu, and Zoubin Ghahramani. Variational inference for nonparametric multiple clustering. In *KDD10 Workshop on Discovering, Summarizing, and Using Multiple Clusterings*, 2010.
- Sonia Jain and Radford M. Neal. A split-merge Markov chain Monte Carlo procedure for the Dirichlet process mixture model. *Journal of Computational and Graphical Statistics*, 13(1):158–182, 2004.
- Vikash Mansinghka, Richard Tibbetts, Jay Baxter, Pat Shafto, and Baxter Eaves. BayesDB: A probabilistic programming system for querying the probable implications of data. *arXiv preprint arXiv:1512.05006*, 2015.
- Vikash Mansinghka, Patrick Shafto, Eric Jonas, Cap Petschulat, Max Gasner, and Joshua B. Tenenbaum. Crosscat: A fully Bayesian nonparametric method for analyzing heterogeneous, high dimensional data. *Journal of Machine Learning Research*, 17(138):1–49, 2016.
- Radford M. Neal. Markov chain sampling methods for Dirichlet process mixture models. *Journal of Computational and Graphical Statistics*, 9(2):249–265, 2000.
- Fritz Obermeyer and Jonathan Glidden. Loom: Streaming cross-categorization engine, 2014. URL <https://github.com/posterior/loom>.
- Fritz Obermeyer, Jonathan Glidden, and Eric Jonas. Scaling nonparametric Bayesian inference via subsample-annealing. *arXiv preprint arXiv:1402.5473*, 2014.
- Du Phan, Neeraj Pradhan, and Martin Jankowiak. Composable effects for flexible and accelerated probabilistic programming in NumPyro. *arXiv preprint arXiv:1912.11554*, 2019.
- Abel Rodriguez, David B. Dunson, and Alan E. Gelfand. The nested Dirichlet process. *Journal of the American Statistical Association*, 103(483):1131–1154, 2008.
- Yee Whye Teh, Michael I. Jordan, Matthew J. Beal, and David M. Blei. Hierarchical Dirichlet processes. *Journal of the American Statistical Association*, 101(476):1566–1581, 2006.

A Inference Query Reference

Table 5 lists all packed inference queries provided by JAX-CROSSCAT.

Table 5: Packed inference queries in JAX-CROSSCAT.

Function	Description
<code>packed_predictive_probability</code>	Log-probability of a value given context
<code>packed_predictive_sample</code>	Sample from posterior predictive
<code>packed_predictive_cdf</code>	Empirical CDF via MC sampling
<code>packed_anomaly_score</code>	Row anomalousness via avg. log-prob
<code>packed_impute_and_confidence</code>	Point estimate + confidence
<code>packed_credible_interval</code>	Percentile-based intervals
<code>packed_mutual_information</code>	MC estimate of mutual information
<code>packed_dependence_probability</code>	View co-assignment probability
<code>packed_dependence_matrix</code>	Full Z-matrix (pairwise)
<code>packed_row_similarity</code>	Category co-assignment probability
<code>packed_row_typicality</code>	Typicality under model
<code>packed_column_typicality</code>	Column typicality
<code>packed_conditional_entropy</code>	Conditional entropy estimate
<code>packed_joint_predictive_probability</code>	Joint log-probability
<code>packed_sample_and_insert</code>	Sample new row and insert
<i>Batch-vectorized variants (vmapped over rows/columns):</i>	
<code>batch_classify_column</code>	Classify all rows for a target column
<code>batch_score_columns_binary</code>	$P(\text{col} = 1)$ for multiple binary columns
<code>batch_anomaly_score</code>	Anomaly scores for all rows
<code>batch_impute_column</code>	Impute a column for multiple rows
<code>batch_row_similarity</code>	Pairwise row similarity matrix
<code>batch_row_typicality</code>	Typicality for multiple rows
<code>batch_predictive_cdf</code>	CDF for multiple rows
<code>batch_credible_interval</code>	Credible intervals for multiple rows
<i>Multi-chain wrappers (aggregate across chains):</i>	
<code>multi_chain_predictive_probability</code>	Averaged log-prob across chains
<code>multi_chain_predictive_sample</code>	Sample from chain mixture
<code>multi_chain_anomaly_score</code>	Averaged anomaly across chains
<code>multi_chain_impute_and_confidence</code>	Averaged imputation
<code>multi_chain_predictive_cdf</code>	Averaged CDF across chains